

SouvenirPOS — Implementation Report

Course: IT342-G01 — Systems Integration and Architecture 1

Project: SouvenirPOS — A Cross-Platform Sales Point-of-Sale System for Souvenir Shops

Prepared by: Ritchie Jay D. Erag

Core Feature: Process Sale (Record a Sale Transaction)

GitHub Repository: <https://github.com/chieeng/SouvenirPOS>

1. Core Feature Overview

Purpose

The core feature of SouvenirPOS is **Process Sale** — recording a complete sales transaction at the counter. It is the central use case of the system (the "Process Sale" use case in the SRS Use Case model, which *includes* "Generate Receipt"), and every other feature exists to support it: authentication protects it, category management feeds it, and sales history and the dashboard read back what it produces.

The feature lets a cashier build a sale from one or more **category + price + quantity** lines, enter the customer's **cash payment**, and complete the transaction. The backend — the single source of truth — **recomputes** every monetary value, **validates** the business rules, **persists** the sale immutably to the shared Supabase database, and returns a **digital receipt**. Because both the ReactJS web app and the Android Kotlin app talk to the same Spring Boot API, a sale rung up on one platform appears on the other within five seconds.

Why it matters

It replaces manual notebook recording with a consistent, online record of every sale that the owner can review remotely and that a second cashier can add to from another device during tour-bus rushes.

2. Functional Requirements Supporting the Core Feature

ID	Requirement	Role in Process Sale
FR-005	Select Sale Category	Cashier picks a category from the owner-managed list for each line
FR-006	Create Sale Transaction	A new sale is opened, tied to the logged-in cashier and current date/time
FR-007	Add Items to Sale	One or more lines, each = category + manually entered price + quantity ≥ 1
FR-008	Compute Sale Totals	Server computes each line subtotal and the sale total
FR-009	Record Payment & Compute Change	Server accepts cash tendered and computes change due
FR-010	Generate Digital Receipt	Completed sale is returned and shown as a receipt
FR-011	Save Sale to Database	Sale + all line items persisted to the shared database
FR-016	Cross-Platform Data Sync	The saved sale is visible on the other platform within 5 s
FR-017 / NFR-004	Access control	Only an authenticated user (valid JWT) may record a sale

Business rules enforced: BR-001 (JWT required), BR-002 (≥ 1 line item), BR-003 (payment $\geq$ total), BR-004 (sale linked to cashier), BR-005 (entered price recorded per line), BR-007 (peso, 2 decimals), BR-008 (saved sale is immutable).

3. Core Feature Workflow

```
Login (JWT)
|
v
Open new sale --> [ Select category -> enter price -> enter quantity -> Add line ] <--
```

```

|
|
| more items? -----+
|
v
Enter cash payment
|
v
Client sends POST /api/sales (items + paymentAmount + JWT)
|
v
Server: authenticate -> validate (BR-002/003) -> recompute subtotals/total/change (FR-008/009)
|
v
Persist Sale + SaleItems to Supabase (FR-011, immutable)
|
v
Return SaleResponse --> Client shows digital receipt (FR-010)
|
v
Sale appears in Sales History & Dashboard on web and mobile within 5 s (FR-016)

```

4. Sequence of Actions

#	User action	System response (client)	Backend processing	Database operation	Expected output
1	Cashier logs in	Stores JWT; opens POS screen	POST /api/auth/login verifies password hash, issues JWT	SELECT user by username	POS screen ready, categories loaded
2	Selects a category	Highlights the chosen category chip	GET /api/categories (on load)	SELECT all categories	Owner-managed category list shown (FR-005)
3	Types a price, sets quantity, taps Add to sale	Adds a line to the cart; running total updates	— (held in client state until checkout)	—	Cart line: category, price, qty, subtotal (FR-007)
4	Repeats for more items	Cart grows; total recalculated for display	—	—	Multi-line cart with live total (FR-008)
5	Enters cash tendered, taps Checkout / Complete sale	Sends POST /api/sales with items + payment + Authorization: Bearer <JWT>	JwtAuthFilter validates token → SaleService.createSale()	—	Request accepted for processing
6	(waits)	Shows spinner	Resolve cashier (BR-004); @Valid checks BR-002 & price/qty; recompute subtotal = unitPrice × quantity, sum total; enforce payment ≥ total (BR-003); change = payment - total (FR-009)	SELECT category by id (per line)	Server-computed, trustworthy totals
7	(waits)	—	Persist the sale graph in one transaction	INSERT into sales + cascade INSERT into sale_items	Sale saved with generated id (FR-011)
8	Sees confirmation	Renders the digital receipt (items, qty, price, total, payment, change)	Returns 201 Created + SaleResponse	—	Digital receipt displayed (FR-010)
9	Owner opens Dashboard /	Auto-refreshing views poll the API	GET /api/sales, GET /api/sales/summary	SELECT sales (optionally by date)	New sale visible within 5 s (FR-012/015/016)

#	User action	System response (client)	Backend processing	Database operation	Expected output
	History (any device)				

Error paths: invalid/expired JWT → 401 ; empty cart or bad price/quantity → 400 with a validation message; payment < total → 400 "Payment amount must be greater than or equal to the total" ; unknown category → 400 "Category not found" .
The client shows the returned message and the sale is **not** saved.

5. Implementation Explanation

5.1 Backend (Spring Boot — single source of truth, NFR-011)

Component	File	Responsibility
SaleController	backend/.../sale/controller/SaleController.java	REST endpoints under /api/sales . createSale() handles POST (@Valid @RequestBody , returns 201 Created); also GET list (FR-012, with optional ?date), GET /summary (FR-015), GET /{id} (FR-014).
SaleService	backend/.../sale/service/SaleService.java	The heart of the feature. createSale() is @Transactional : resolves the authenticated cashier (currentUser() , BR-004), loops the lines, looks up each Category , computes subtotal = unitPrice × quantity and the running total with BigDecimal (FR-008, BR-007), enforces payment ≥ total (BR-003), sets change = payment - total (FR-009), and saves.
Sale (entity)	backend/.../shared/domain/Sale.java	JPA @Entity → sales table. Holds saleDateTime , cashier (@ManyToOne , not null — BR-004), totalAmount , paymentAmount , changeAmount , and a cascaded @OneToMany list of SaleItem .
SaleItem (entity)	backend/.../shared/domain/SaleItem.java	JPA @Entity → sale_items table. Holds category , quantity , unitPrice (BR-005 — the price entered at sale time), and subtotal , all precision=12, scale=2 (BR-007).
SaleRepository	backend/.../sale/repository/SaleRepository.java	Spring Data JPA repository; derived queries for date-range and ordered listing.
SaleCreateRequest	backend/.../sale/dto/SaleCreateRequest.java	Request DTO. @NotEmpty on items enforces BR-002 (≥1 line); @NotNull

Component	File	Responsibility
		<code>paymentAmount</code> ; <code>@Valid</code> cascades to each line.
<code>SaleItemRequest</code>	<code>backend/.../sale/dto/SaleItemRequest.java</code>	Per-line DTO. <code>@NotNull</code> <code>categoryId</code> , <code>@Min(1)</code> <code>quantity</code> (FR-007), <code>@DecimalMin("0.01")</code> <code>unitPrice</code> (price > 0).
<code>SaleResponse</code> / <code>SaleItemResponse</code>	<code>backend/.../sale/dto/</code>	Response DTOs used to build the digital receipt payload (FR-010).
<code>GlobalExceptionHandler</code>	<code>backend/.../shared/exception/GlobalExceptionHandler.java</code>	<code>@RestControllerAdvice</code> : validation errors → 400 (field → message); <code>IllegalArgumentException</code> (payment < total, category not found) → 400 ; bad credentials → 401 ; access denied → 403 .
<code>JwtAuthFilter</code> / <code>JwtService</code> / <code>SecurityConfig</code>	<code>backend/.../shared/security/</code>	Enforce BR-001 / NFR-004: every <code>/api/**</code> call (except login) requires a valid JWT; the filter validates the token and puts the user in the <code>SecurityContext</code> so <code>SaleService</code> can attribute the sale.

5.2 Web client (ReactJS — owner/desktop)

Component	File	Responsibility
<code>POSPage</code>	<code>web/src/pos/components/POSPage.jsx</code>	The sale screen. React state holds <code>categories</code> , selected <code>category</code> , keypad <code>priceInput</code> , <code>quantity</code> , <code>cart</code> , and <code>payment</code> . <code>handleAddToCart()</code> validates <code>category + price > 0 + quantity ≥ 1</code> and appends a line; <code>handleCheckout()</code> builds the request and <code>POST</code> s <code>/sales</code> , then stores the returned sale as <code>receipt</code> for the receipt modal (FR-010). Totals/change are computed client-side for display only .
<code>client</code> (Axios)	<code>web/src/shared/api/client.js</code>	<code>baseUrl</code> : <code>'/api'</code> + a request interceptor that attaches <code>Authorization: Bearer <token></code> (BR-001).
<code>AuthContext</code>	<code>web/src/auth/context/AuthContext.jsx</code>	Stores the JWT and user after login and exposes it to the POS.

5.3 Mobile client (Android Kotlin / Jetpack Compose — cashier)

Component	File	Responsibility
<code>PosScreen</code>	<code>mobile/.../pos/ui/PosScreen.kt</code>	The three-step sale UI: Sell (category chips, amount keypad, quantity stepper), Cart (line items with per-line quantity steppers, subtotal/total), Payment (amount due, quick-cash chips, cash keypad, change due). <code>ReceiptDialog</code> shows the digital receipt (FR-010).
<code>PosViewModel</code>	<code>mobile/.../pos/viewmodel/PosViewModel.kt</code>	Holds the same state as the web page (<code>categories</code> , <code>priceInput</code> , <code>quantity</code> , <code>cart</code> of <code>CartLine</code> , <code>payment</code>). <code>addToCart()</code> mirrors FR-007 validation; <code>canCheckout</code> enforces BR-002/BR-003 before enabling checkout; <code>checkout()</code> calls the repository and exposes the returned <code>SaleResponse</code> .

Component	File	Responsibility
PosRepository / ApiService	mobile/.../pos/data/PosRepository.kt , mobile/.../core/network/ApiService.kt	Retrofit call to <code>POST /api/sales</code> .
AuthInterceptor	mobile/.../core/network/AuthInterceptor.kt	Attaches the JWT to every request (BR-001).

5.4 How data is validated, processed, stored, and displayed

- **Validated (two layers).** The clients pre-validate for a smooth UX (a line needs a category, a price > 0, and quantity ≥ 1; checkout is disabled until the cart is non-empty and `payment ≥ total`). The **server re-validates** authoritatively with Bean Validation (`@Valid` , `@NotEmpty` , `@Min(1)` , `@DecimalMin`) plus the explicit `payment ≥ total` check — so a malformed or malicious request is rejected regardless of the client (NFR-011).
- **Processed.** All money is handled as `BigDecimal` at scale 2 (BR-007). The server **recomputes** `subtotal` , `total` , and `change` and never trusts client-sent totals (FR-008/009).
- **Stored.** `SaleService.createSale()` runs in one `@Transactional` unit; JPA cascades the `Sale` and its `SaleItem` s into the `sales` and `sale_items` tables in the shared Supabase (PostgreSQL) database (FR-011). There is no update/delete endpoint, so a saved sale is immutable (BR-008).
- **Displayed.** The server returns a `SaleResponse` , which the client renders as a digital receipt (FR-010). The same records are read back by Sales History (`GET /api/sales`) and the Dashboard (`GET /api/sales/summary`); both clients poll every 5 s, satisfying the cross-platform sync window (FR-016).

6. Commit History Table

Each major task / functional requirement has its own commit. Links use the repository <https://github.com/chieeng/SouvenirPOS>.

Task / Requirement	Description	Commit
Project scaffold (JWT auth, user mgmt, POS UI)	Initial cross-platform scaffold: JWT authentication, admin user management, POS screen (FR-001/002/003)	c7e6159
Process Sale — API + wiring	Add sales/category API, Android app, and wire the web POS to the backend (FR-005–FR-011)	631878c
Owner-only Manage Users (mobile)	Add owner-only Manage Users screen to Android (FR-001/004/017)	45e3378
View Sales History (web)	Sales history list, date filter, detail (FR-012/013/014)	64e1c40
View Sales History (mobile)	Sales history screen on Android (FR-012/013/014)	6645bb4
Refactor sale into vertical slice	Reorganize sales processing into a feature slice	ff8b239
Categories + Dashboard features	Add owner Category management and Dashboard sales summary (FR-015/019)	b7ebea0
UI redesign (web)	Apply the Harbor Row design system to the web app	9cfdaed
Mobile theme + login redesign	Teal/amber theme and login screen on Android	f6bb9c8
Mobile POS flow rebuild	Sell → Cart → Payment flow with bottom navigation	1a9dec1
SRS compliance — restore quantity	Restore per-line quantity input to match FR-007 / BR-002 / ERD	181c6aa

Full history: `git log --oneline` or the repository's [commits page](#).

7. Traceability Note (SRS alignment)

The implemented Process Sale feature is consistent with SRS v2.0: FR-005 through FR-011 are all realized, the server enforces BR-001–BR-008, and the ERD (`User` → `Sale` → `SaleItem` → `Category`) is mirrored one-to-one by the JPA entities. During the UI redesign the per-line **quantity** input was briefly removed; because FR-007, BR-002, the Activity and Sequence diagrams, and

`SaleItem.quantity` in the ERD all require it, the quantity control was restored on both clients (commit `181c6aa`) so the running system matches the SRS.